

Tripppeltopppar

Fakta: Det finns berg i Bolivia □ (denna fakten har granskats av riktiga Bolivianska patrioter).
Bergsväg, ta mig hem, till stället, där jag tillhör, västra o____n, landsmamma, bergvsägar.

I Bolivia finns en sekvens av N bergstoppar, numrerade från 0 till $N - 1$. **Höjden** för toppen i ($0 \leq i < N$) är $H[i]$, vilket är ett heltal mellan 1 och $N - 1$.

För två toppar i och j där $0 \leq i < j < N$, definieras **avståndet** mellan dem som $d(i, j) = j - i$.

Joshua har nu bestämt att en tripplett av toppar **susig** om den har följande speciella egenskap:
Höjderna på de tre topparna **matchar** deras parvisa avstånd **utan att beakta ordningen**.

Formellt sett är en tripplett av index (i, j, k) susig om

- $0 \leq i < j < k < N$, och
- höjderna $(H[i], H[j], H[k])$ matchar de parvisa avstånden $(d(i, j), d(i, k), d(j, k))$ om man inte kollar på ordning. Exempelvis, för indexen 0, 1, 2 är de parvisa avstånden $(1, 2, 1)$, så för följande exempelhöjder $(H[0], H[1], H[2]) = (1, 1, 2)$, $(H[0], H[1], H[2]) = (1, 2, 1)$, och $(H[0], H[1], H[2]) = (2, 1, 1)$ hade alla dessa matchat, men höjderna $(H[0], H[1], H[2]) = (1, 2, 2)$ matchar inte dem.

Detta problem består av två delar, där varje deluppgift är associerad med antingen **Del I** eller **Del II**. Du kan lösa deluppgifterna i valfri ordning. I synnerhet, behöver du **inte** slutföra hela Del I innan du försöker dig på Del II.

Del I

Givet en beskrivning av bergskedjan, så är din uppgift att räkna antalet susiga trippletter.

Implementeringsdetaljer

Innan du ställer frågan "hur?", skall du implementera följande procedur:

```
long long count_triples(std::vector<int> H)
```

- H : en array av längd N , som representerar höjderna av topparna 🏔.
- Denna procedur anropas exakt en gång för varje testfall.

Proceduren ska returnera ett heltal T , antalet susiga trippletter i bergskedjan.

Begränsningar

- $3 \leq N \leq 200\,000$
- $1 \leq H[i] \leq N - 1$ för varje i där $0 \leq i < N$.

Subtasks

Del I är sammanlagt värd 70 poäng.

Subtask	Poäng	Ytterligare begränsningar
1	8	$N \leq 100$
2	6	$H[i] \leq 10$ för varje i sådan att $0 \leq i < N$.
3	10	$N \leq 2000$
4	11	Höjderna är icke-minskande. Det vill säga, $H[i - 1] \leq H[i]$ för varje i sådan att $1 \leq i < N$.
5	16	$N \leq 50\,000$
6	19	Inga ytterligare begränsningar.

Exempel

Pondera följande skrik.

```
count_triples([4, 1, 4, 3, 2, 6, 1])
```

Det finns 3 susiga tripletter i bergskedjan:

- För $(i, j, k) = (1, 3, 4)$ matchar höjderna $(1, 3, 2)$ de parvisa avstånden $(2, 3, 1)$.
- För $(i, j, k) = (2, 3, 6)$ matchar höjderna $(4, 3, 1)$ de parvisa avstånden $(1, 4, 3)$.
- För $(i, j, k) = (3, 4, 6)$ matchar höjderna $(3, 2, 1)$ de parvisa avstånden $(1, 3, 2)$.

Därför bör proceduren returnera 3.

Observera att indexen $(0, 2, 4)$ inte bildar en susig tripplett, eftersom höjderna $(4, 4, 2)$ inte matchar de parvisa avstånden $(2, 4, 2)$.

Del II

Din uppgift är att konstruera bergskedjor med många susiga tripletter. Den här delen består av 6 subtasks med **endast utdata (output-only)**, med **delvis poängsättning**.

I varje deluppgift får du två positiva heltal M och K , och du ska konstruera en bergskedja med **högst** M toppar. Om din lösning innehåller **minst** K susiga tripletter får du full poäng för denna

deluppgift. Annars kommer din poäng att vara proportionell mot antalet susiga tripletter som din lösning innehåller.

Observera att din lösning måste bestå av en giltig bergskedja. Anta specifikt att din lösning har N toppar (N måste uppfylla $3 \leq N \leq M$). Då måste höjden på toppen i ($0 \leq i < N$), betecknad med $H[i]$, vara ett heltal mellan 1 och $N - 1$, inklusive.

Implementationsdetaljer

Det finns två metoder för att skicka in din lösning, och du kan använda valfritt mellan dessa för varje deluppgift:

- **Utdatafil**
- **Proceduranrop**

För att skicka in din lösning via en **utdatafil**, skapa och skicka in en textfil i följande format:

```
N
H[0] H[1] ... H[N-1]
```

För att skicka in din lösning via ett **proceduranrop** bör du implementera följande procedur.

```
std::vector<int> construct_range(int M, int K)
```

- M : det maximala antalet toppar.
- K : det önskade antalet tripletter som är susiga.
- Denna procedur anropas exakt en gång för varje deluppgift.

Proceduren ska returnera en array H med längden N , som representerar topparnas höjder.

Subtasks och Poängsättning

Del II är värd totalt 30 poäng. För varje subtask finns endast ett testfall, där värdena för M och K är konstanta och anges i följande tabell:

Subtask	Poäng	M	K
7	5	20	30
8	5	500	2000
9	5	5000	50 000
10	5	30 000	700 000
11	5	100 000	2 000 000
12	5	200 000	12 000 000

För varje subtask, om din lösning inte bildar en giltig bergskedja, blir din poäng 0 (rapporteras som `Output isn't correct` i CMS).

Annars, låt T beteckna antalet susiga tripletter i din lösning. Då blir din poäng för deluppgiften:

$$5 \cdot \min \left(1, \frac{T}{K} \right).$$

Exempeldomare

Del I och II använder samma exempeldomare, där skillnaden mellan de två delarna bestäms av den första raden i inmatningen.

Indataformat för Del I:

```
1
N
H[0] H[1] ... H[N-1]
```

Utdataformat för Del I:

```
T
```

Indataformat för Del II:

```
2
M K
```

Utdataformat för Del II:

```
N
H[0] H[1] ... H[N-1]
```

Observera att utdatan från exempeldomaren matchar formatet som efterfrågas i utdatafilen i del II.